

React + PHP + MySQL

회원가입 & 로그인



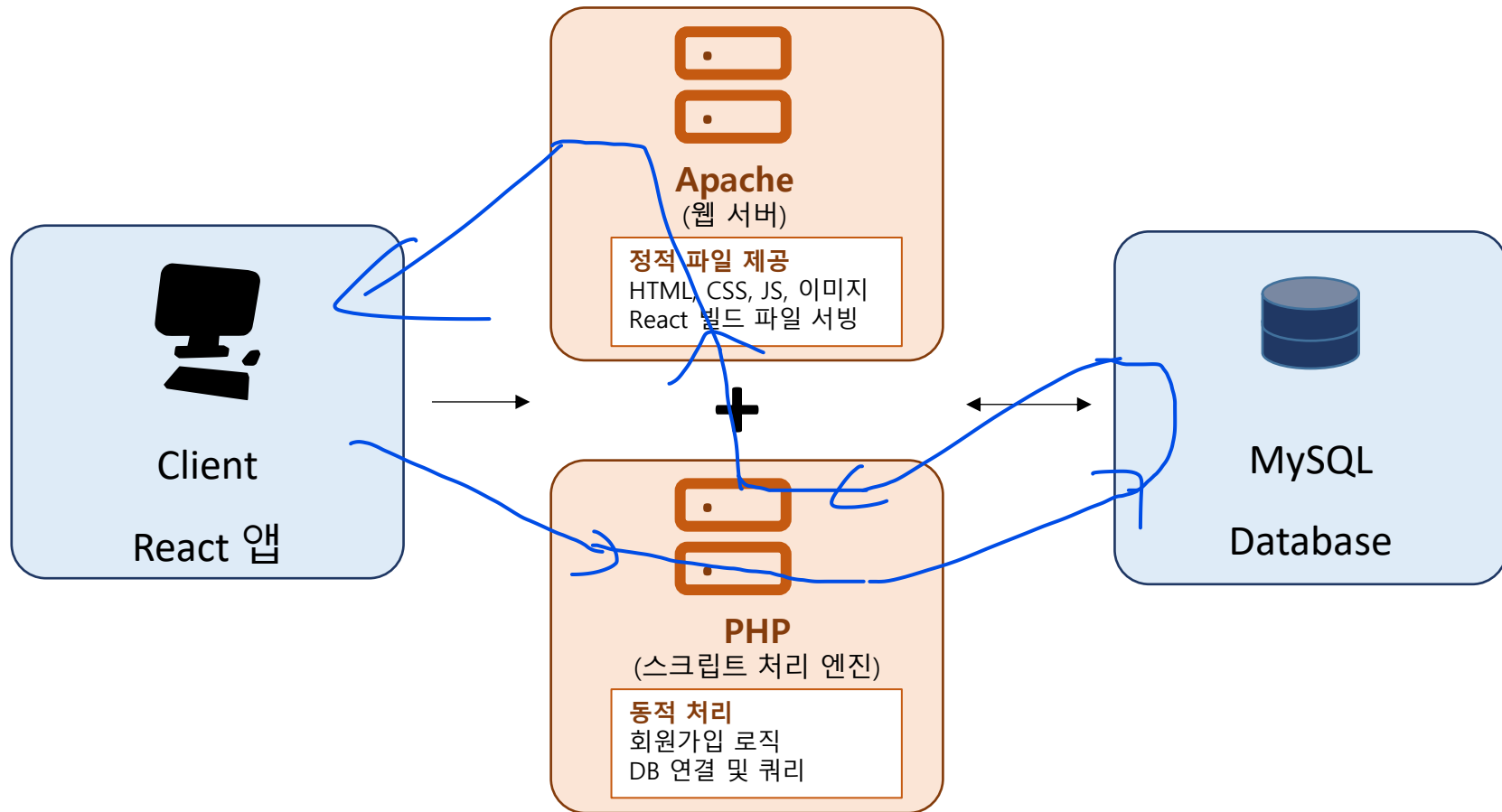


➤ Apache (Apache HTTP Server) 란

- 웹 서버 (Web Server)
- PHP를 실행할 수 있다.
- XAMPP에 포함되어 있다.

➤ XAMPP (PHP 환경)

- 웹 서버로서 정적 파일만 제공하고, PHP는 **별도의 모듈**로서 동적 처리를 담당한다. XAMPP는 이 둘을 **패키지로 묶어서** 제공한다.





➤ 아파치 서버 설치

➤ <https://www.apachefriends.org/>



The screenshot shows the XAMPP Apache Friends website. The header includes navigation links: Apache Friends, Download, Hosting, Community, and About. There is a search bar and a language selector set to EN. The main heading is "XAMPP Apache + MariaDB + PHP + Perl". Below this, a section titled "What is XAMPP?" explains that XAMPP is a popular PHP development environment, free and easy to install, containing MariaDB, PHP, and Perl. To the right is a large XAMPP logo. At the bottom, there are three download buttons: "Download" (with a link to other versions), "XAMPP for Windows 8.2.12 (PHP 8.2.12)" (highlighted with a red box), "XAMPP for Linux 8.2.12 (PHP 8.2.12)", and "XAMPP for OS X 8.2.4 (PHP 8.2.4)".

Apache Friends Download Hosting Community About Search.. Search EN

XAMPP Apache + MariaDB + PHP + Perl

What is XAMPP?

XAMPP is the most popular PHP development environment

XAMPP is a completely free, easy to install Apache distribution containing MariaDB, PHP, and Perl. The XAMPP open source package has been set up to be incredibly easy to install and to use.

Download
Click here for other versions

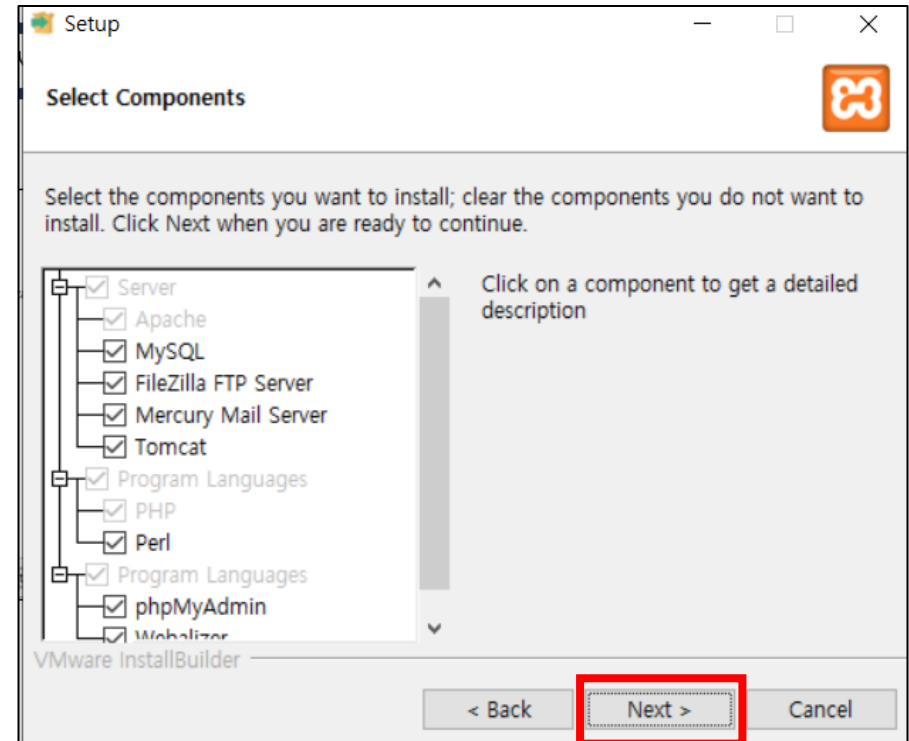
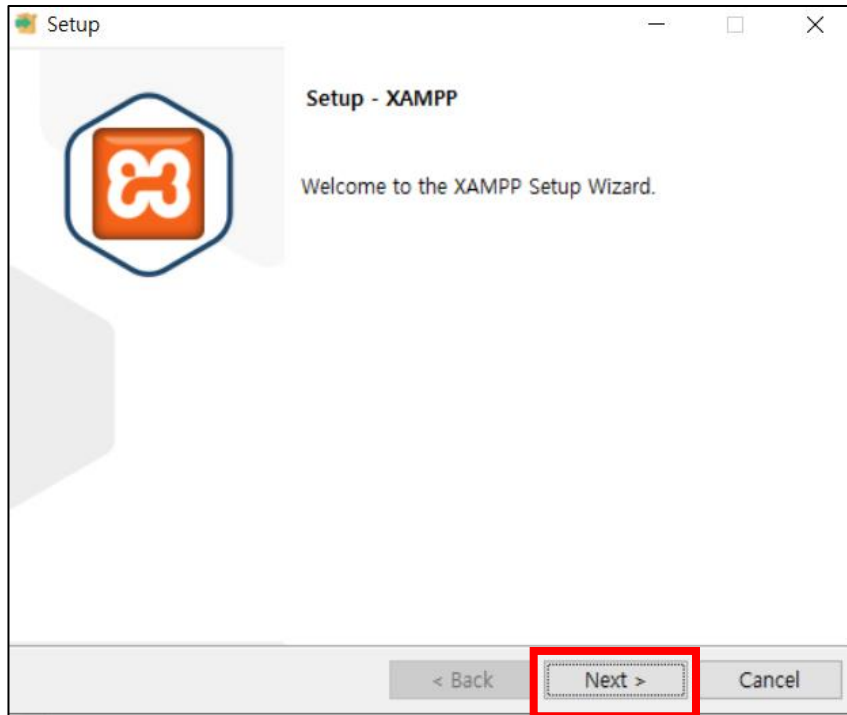
 **XAMPP for Windows**
8.2.12 (PHP 8.2.12)

 **XAMPP for Linux**
8.2.12 (PHP 8.2.12)

 **XAMPP for OS X**
8.2.4 (PHP 8.2.4)

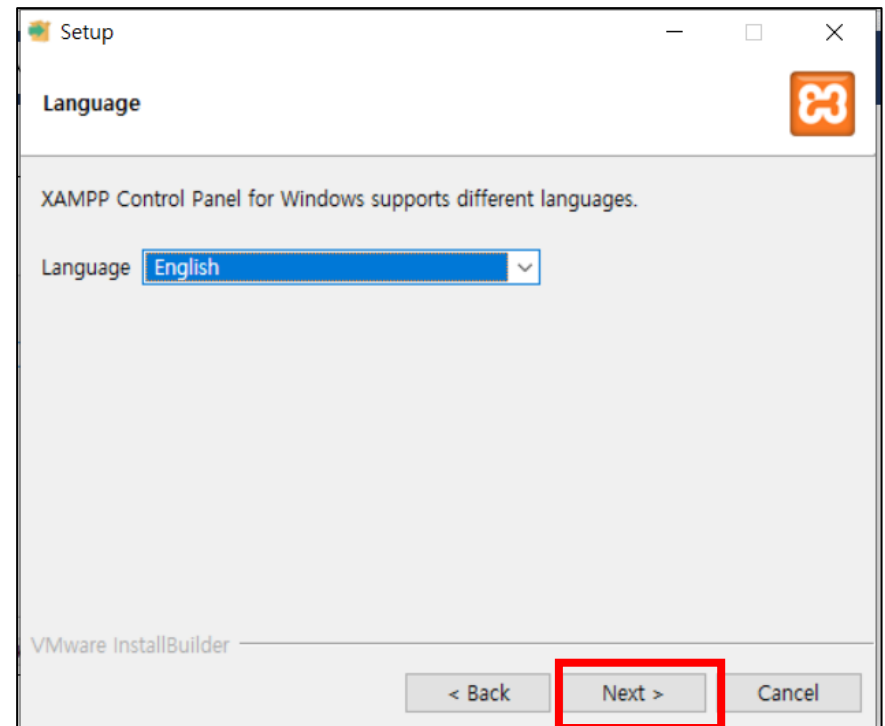
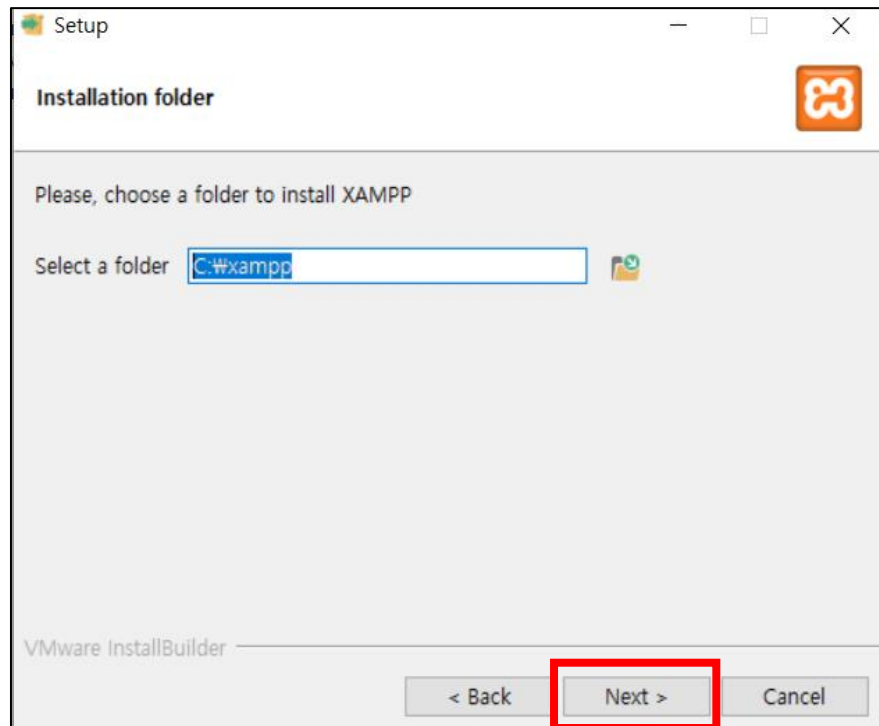


➤ 아파치 서버 설치



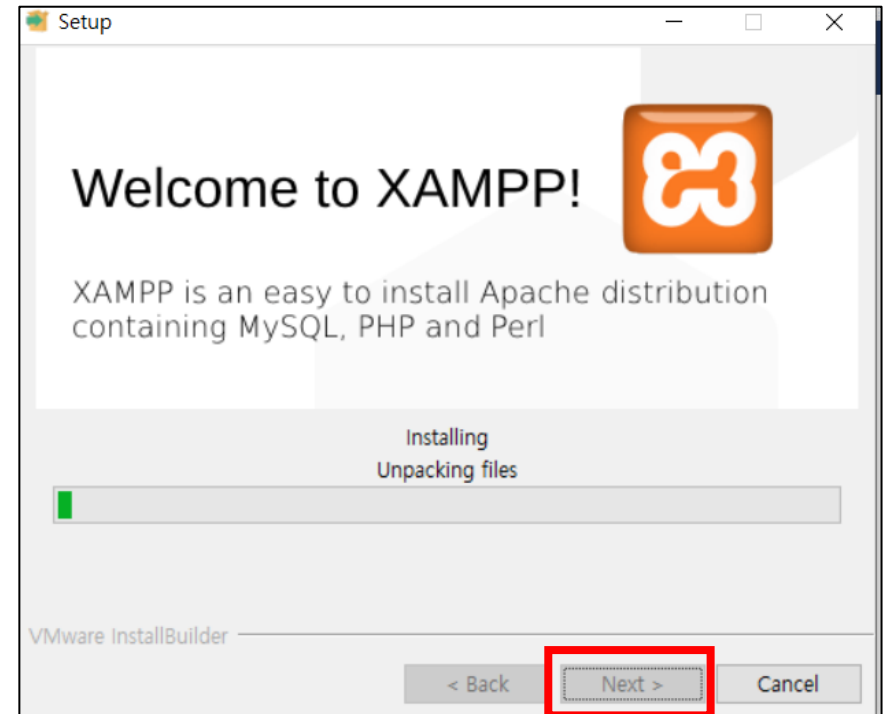
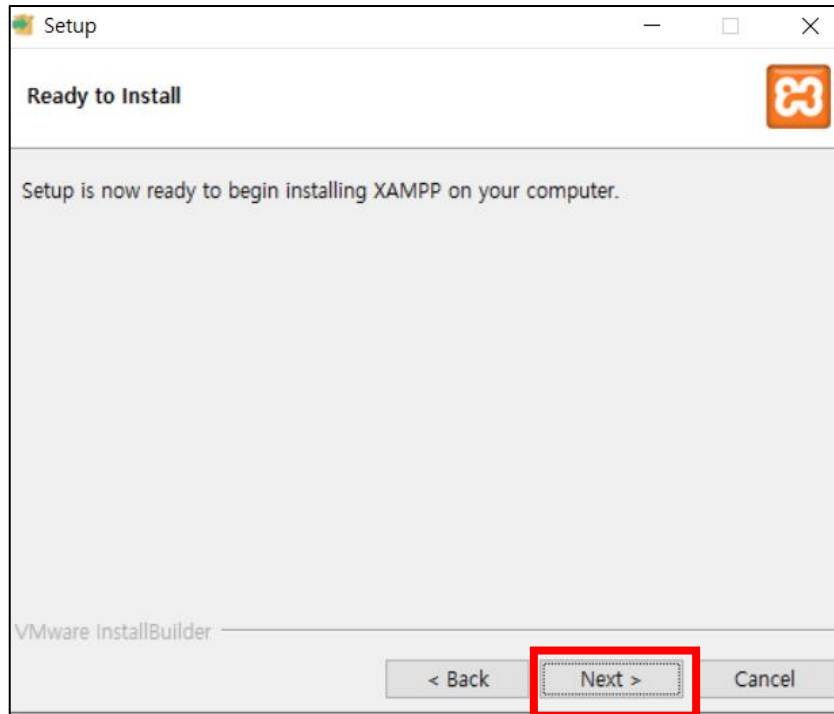


▶ 아파치 서버 설치



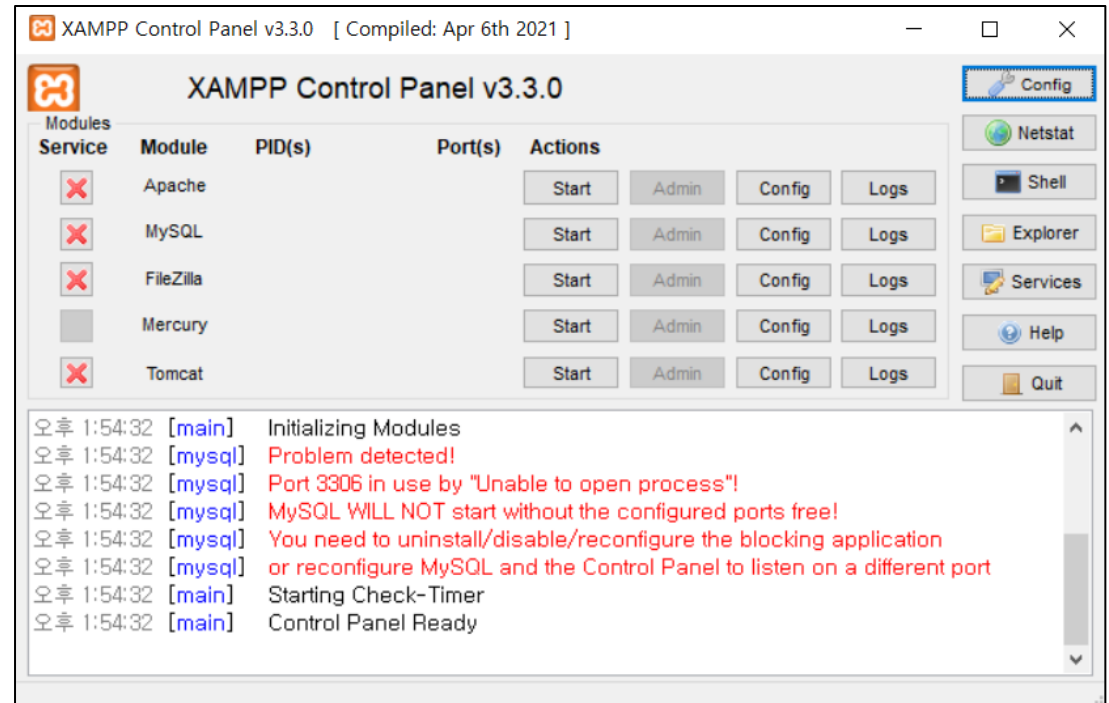
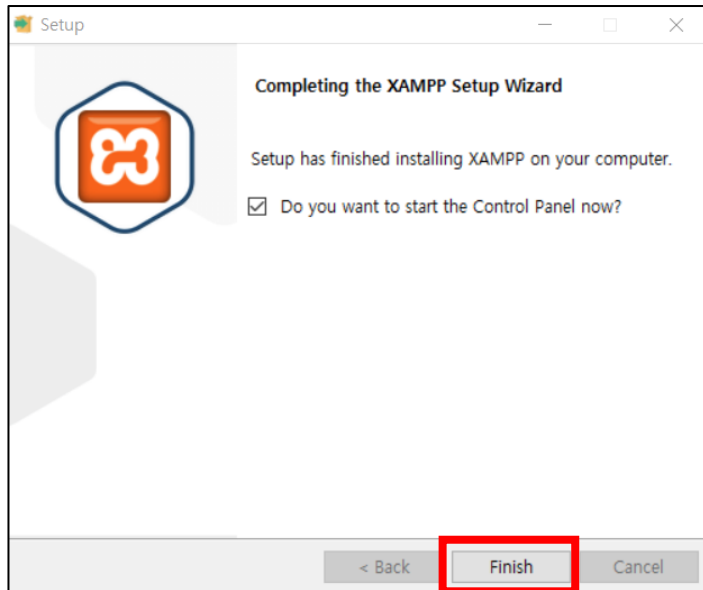


▶ 아파치 서버 설치



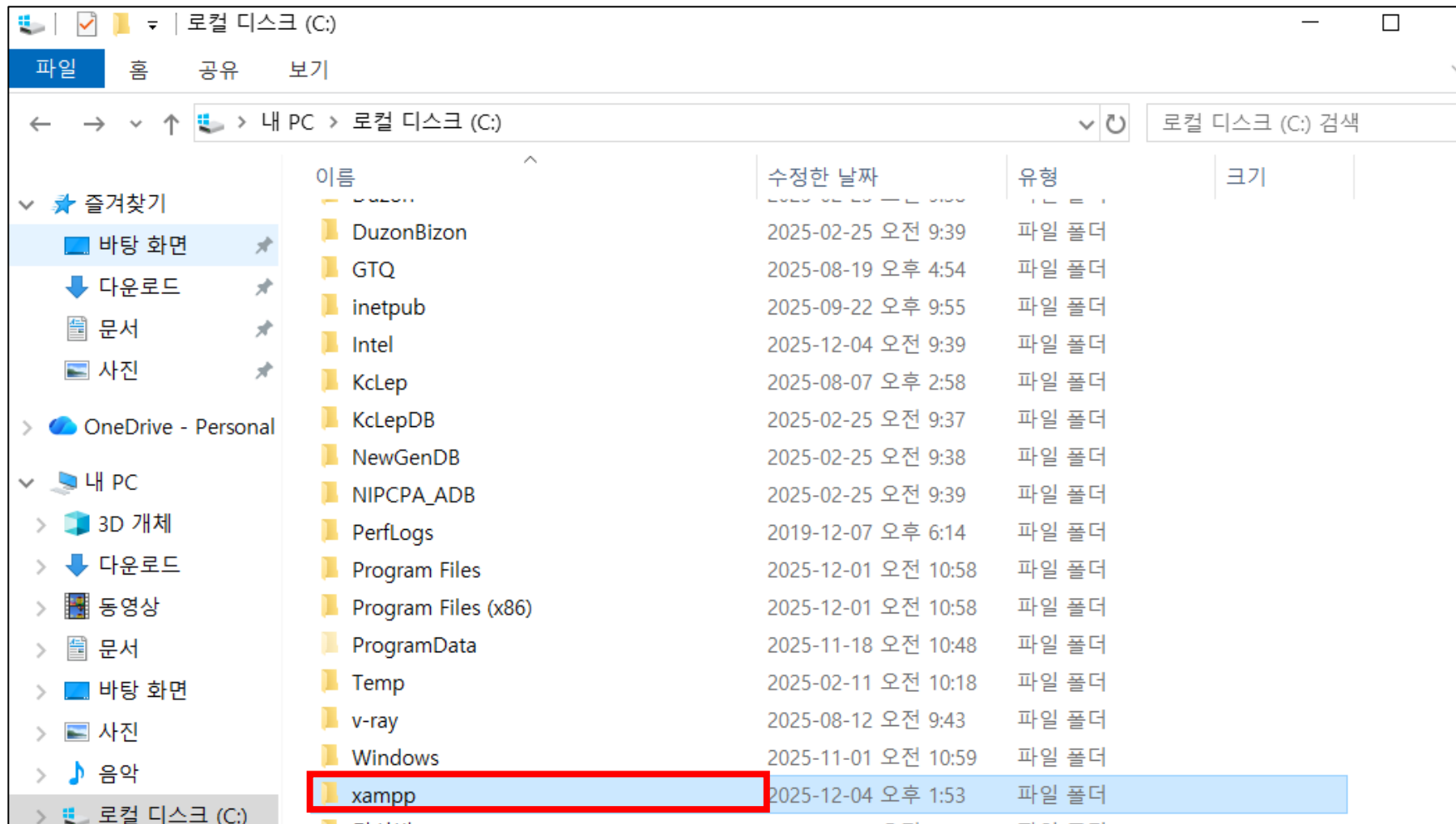


▶ 아파치 서버 설치



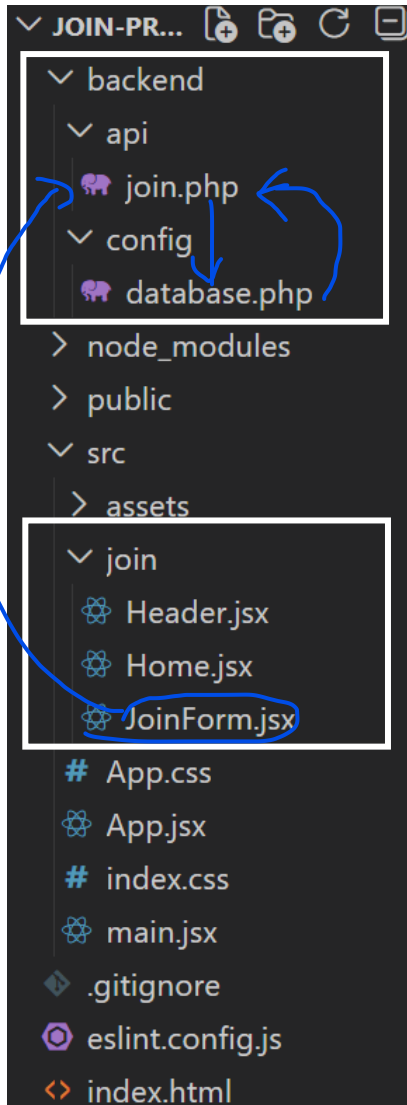


▶ 아파치 서버 설치 후 C:\xampp 생성되었는지 확인





➤ Join-project 폴더 구조를 아래와 같이 생성한다.



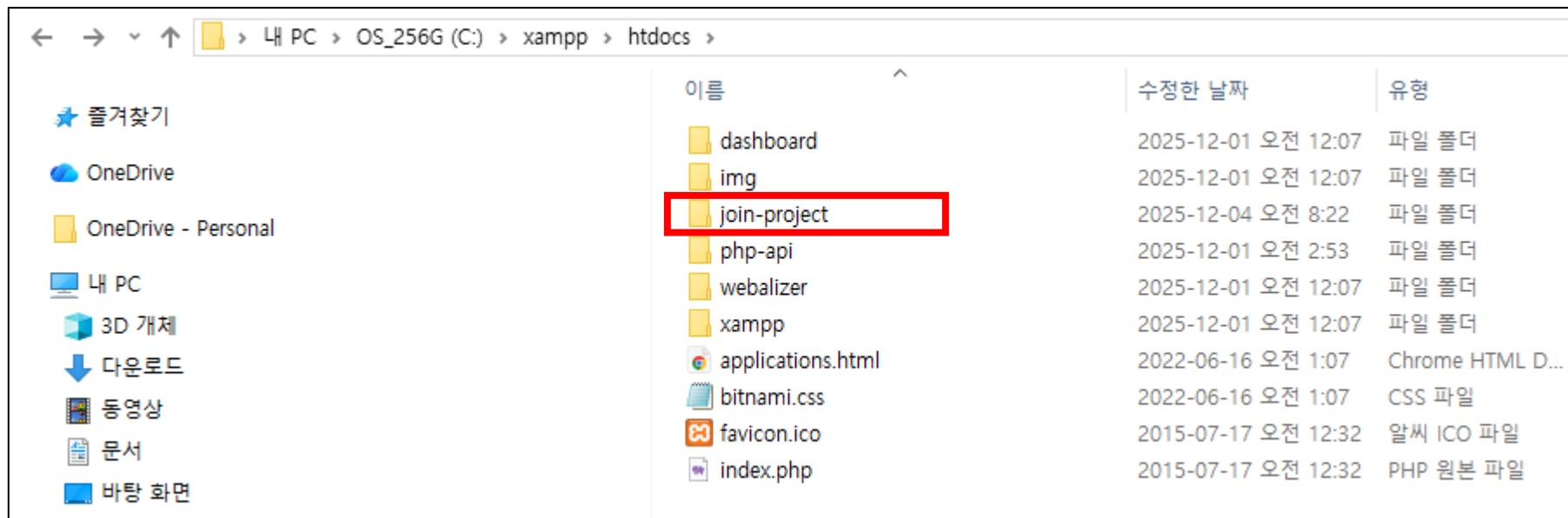


➤ Join-project 폴더를 생성한 후 XAMPP의 기본 웹 루트로 폴더를 통째로 이동한다.

XAMPP의 기본 웹 루트:

Windows: C:\xampp\htdocs\

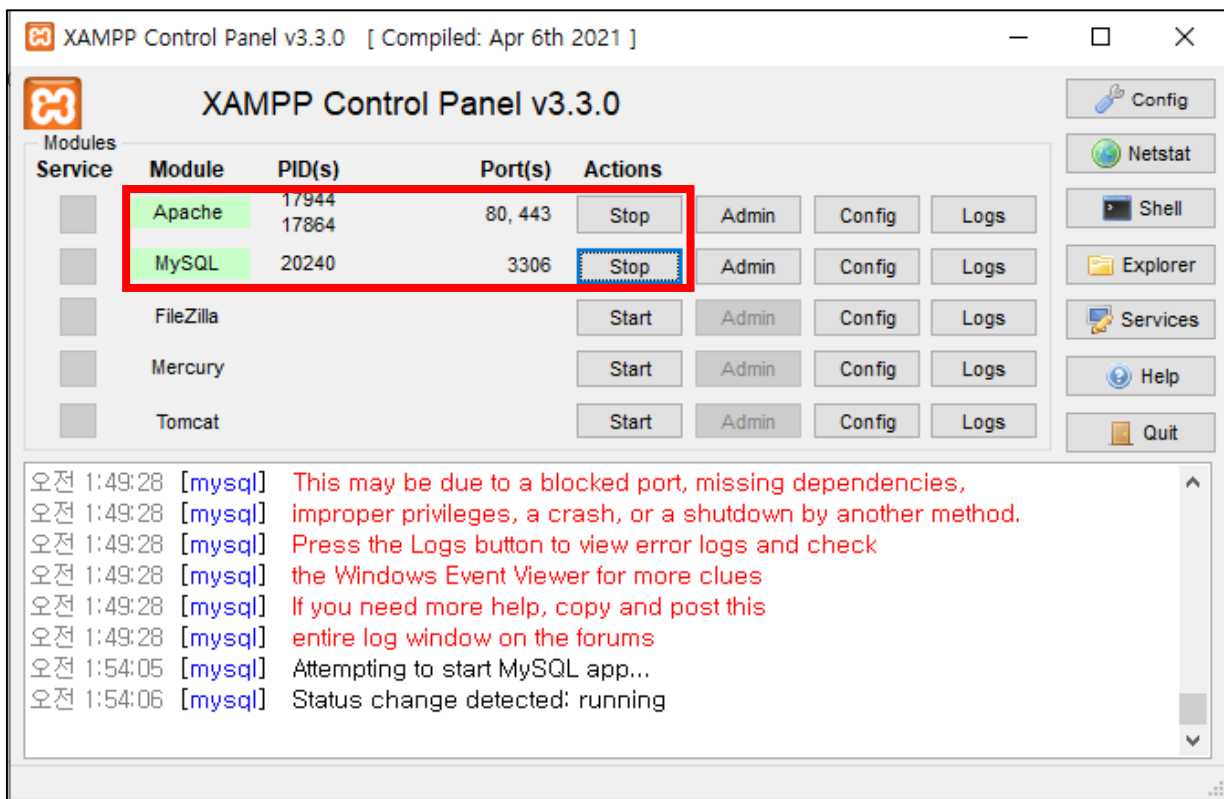
예: C:\xampp\htdocs\join-project => 폴더를 이동한다.





➤ XAMPP에서 Apache와 MySQL 시작

1. XAMPP Control Panel 실행
2. Apache 옆의 Start 버튼 클릭
3. MySQL 옆의 Start 버튼 클릭
4. 둘 다 초록색으로 변하면 정상 실행





➤ XAMPP에서 MySQL 시작 버튼 클릭 후 오류가 출력될 :

- 이유는 이미 3306포트번호인 MySQL Workbench가 실행되어 있기 때문이다.
- 고로 MySQL.exe 파일을 찾아 작업끝내기를 실행해야 한다.

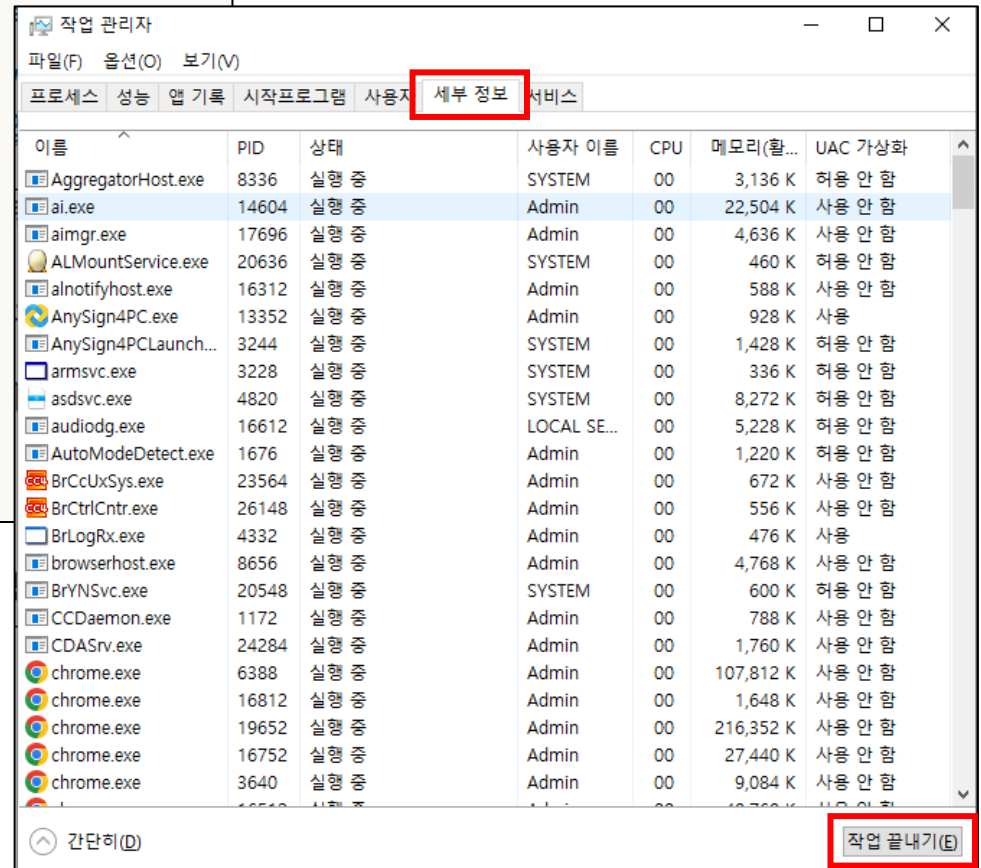
방법 1: 포트 사용 중인 프로그램 확인 및 종료 (추천)

1. 작업 관리자 열기

- **Ctrl + Shift + Esc** 누르기

2. MySQL 관련 프로세스 찾기

- "세부 정보" 탭 클릭
- **mysqld.exe** 검색
- 있다면 작업 끝내기 클릭





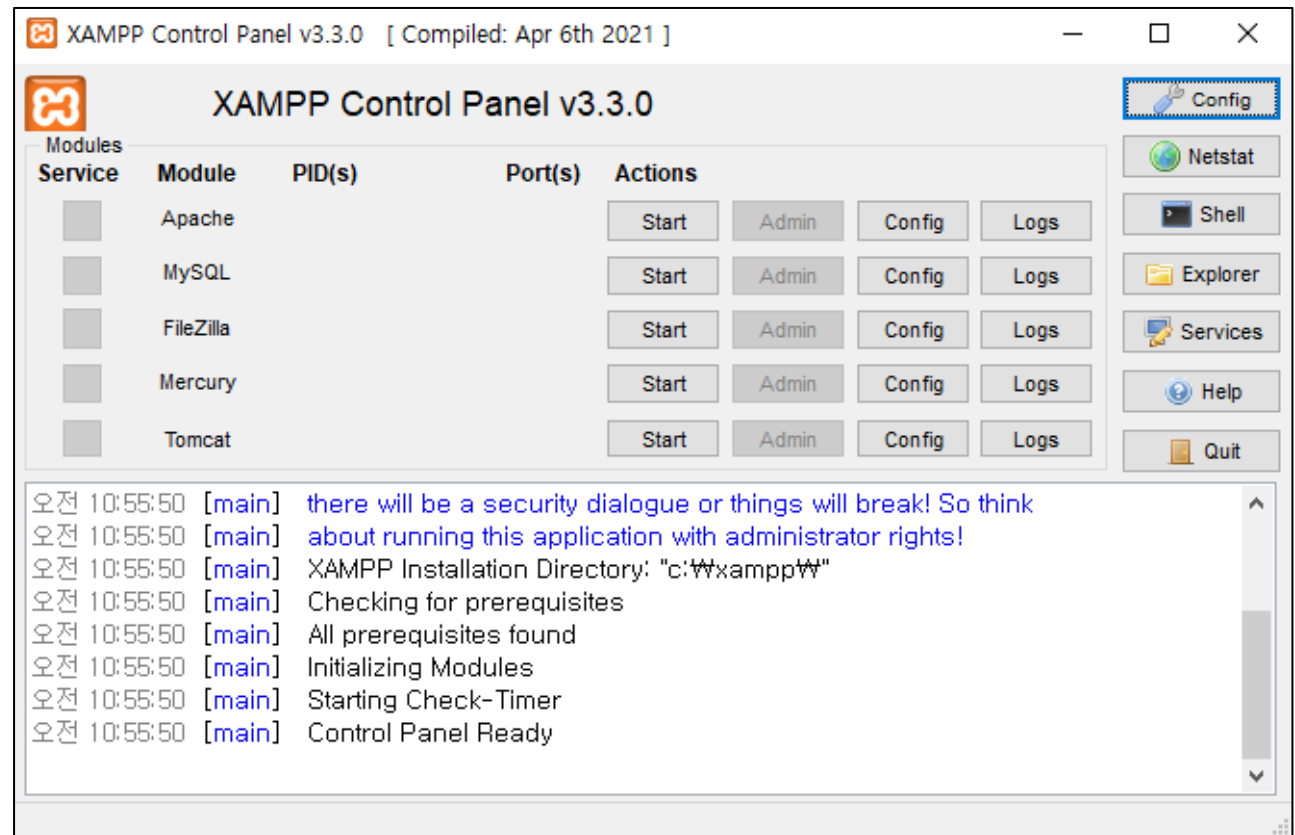
➤ XAMPP 종료 하는 방법

서비스 중지

- Apache [Stop] 버튼 클릭
- MySQL [Stop] 버튼 클릭

➤ XAMPP Control Panel 닫기

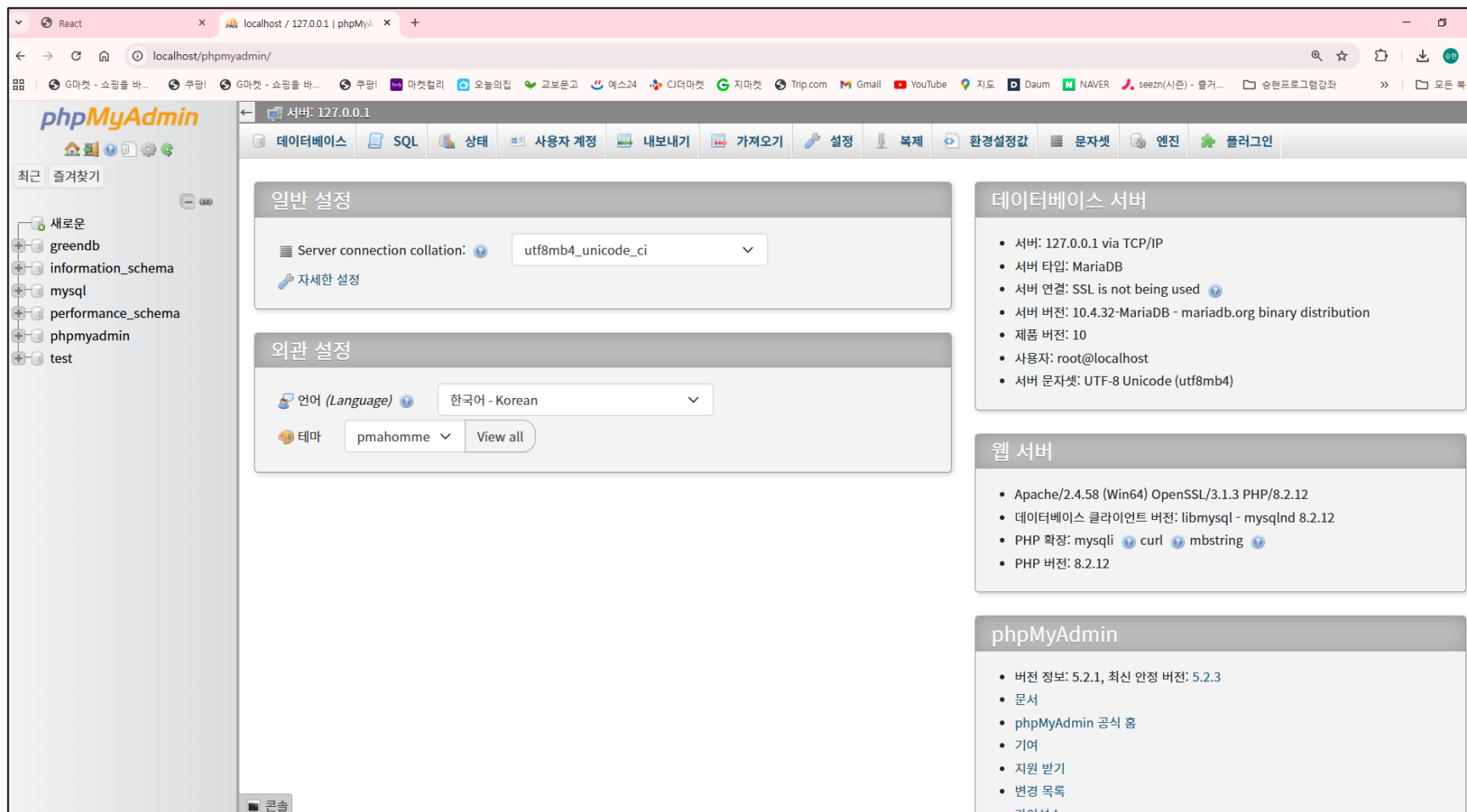
➤ 우측 상단 X 버튼 클릭





➤ phpMyAdmin 접속

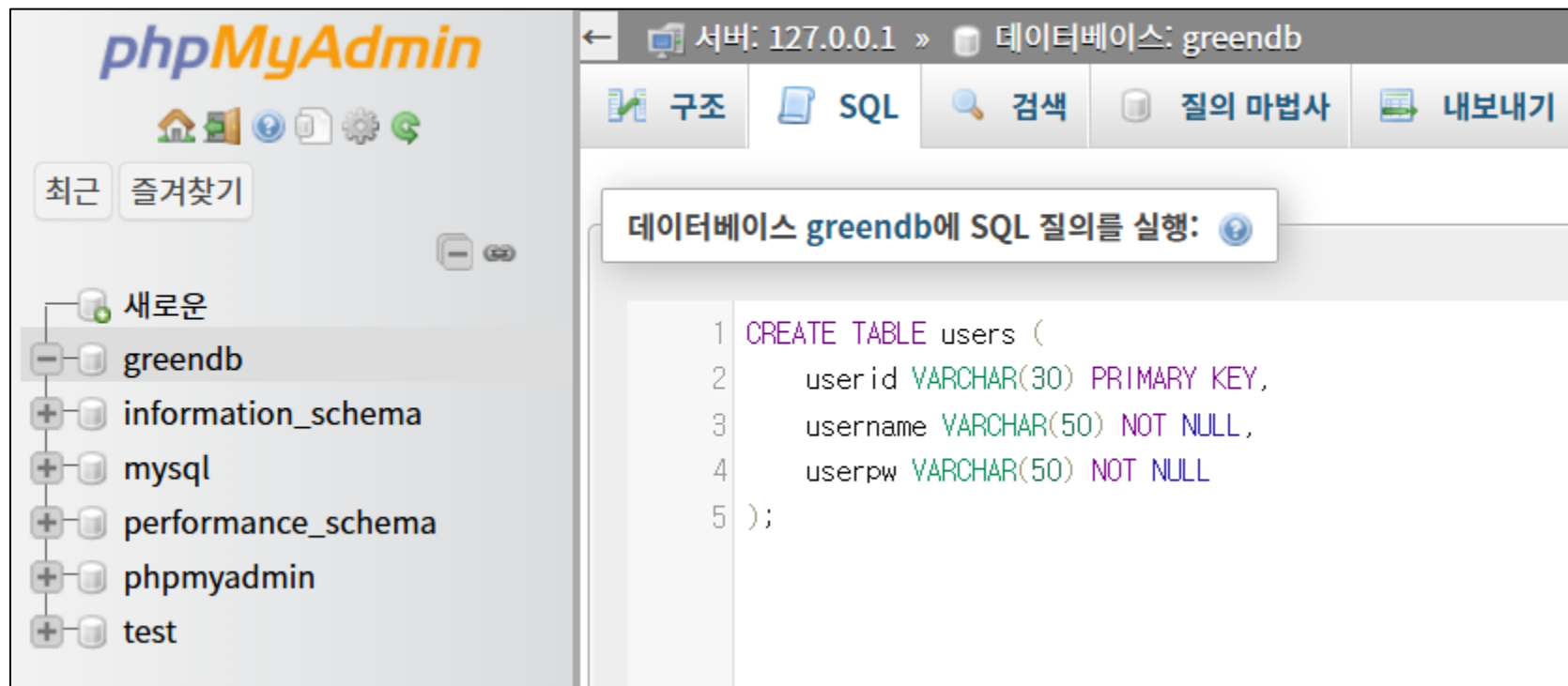
1. 브라우저에서 `http://localhost/phpmyadmin` 접속
2. 왼쪽 "New" 클릭하여 새 데이터베이스 생성





➤ phpMyAdmin 접속

1. SQL 탭 메뉴 클릭 후 아래와 같이 users테이블을 생성한다.





➤ phpMyAdmin 접속

1. 왼쪽 메뉴 greendb 안에 users테이블이 생성되어 있다.

The screenshot shows the phpMyAdmin web interface. On the left sidebar, the 'greendb' database is selected, and the 'users' table is highlighted with a red box. The main panel displays the 'users' table structure. At the top, it shows the server path: '서버: 127.0.0.1 » 데이터베이스: greendb » 테이블: users'. Below this is a toolbar with icons for '보기' (View), '구조' (Structure), 'SQL', '검색' (Search), '삽입' (Insert), '내보내기' (Export), '가져오기' (Import), '권한' (Privileges), and '테이블 작업' (Table Operations). A '질의 상자 보기' (View Query Box) button is present. The main content area shows a green status bar indicating '0 - 1행 표시 중 (합계 2, 질의 실행시간 0.0002 초.)'. Below this is the SQL query: 'SELECT * from users;'. There are links for '프로파일링' (Profiling), '인라인 편집' (Inline Edit), '수정' (Edit), 'SQL 해석' (SQL Explain), 'PHP 코드 보기' (View PHP Code), and '다시 보기' (Refresh). At the bottom, there are controls for '모두 보기' (Show All), '행 갯수' (Number of Rows) set to 25, '행 필터링' (Row Filtering) set to '현재 테이블 검색' (Search current table), and 'Sort by key' set to '없음' (None). An 'Extra options' button is also visible.

회원가입 JSX





```
import { useEffect, useState } from 'react';
import { Link } from 'react-router-dom';
//import api from './axios';
import './Header.css';
export default function Header() {
  return (
    <header className="header">
      <h2>
        <Link to="/" className="logo">
          { ' ' }
          React 쇼핑몰
        </Link>
      </h2>
      <nav className="nav">
        <Link to="/">Home</Link>
        <Link to="/join">Join</Link>
      </nav>
    </header>
  );
}
```

```
.header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 1rem 2rem;
  background-color: #fff;
  border-bottom: 1px solid #e0e0e0;
}

.logo {
  color: #333;
  text-decoration: none;
  font-weight: bold;
  font-size: 1.5rem;
}

.logo:hover {
  color: #0066cc;
}

.nav {
  display: flex;
  gap: 1.5rem;
}

.nav a {
  color: #666;
  text-decoration: none;
  font-size: 1rem;
  transition: color 0.2s;
}

.nav a:hover {
  color: #0066cc;
}
```



React 쇼핑몰

Home

Join



```
import './JoinForm.css';

export default function JoinForm() {

  return (
    <div className="joinWrap">
      <h2> 회원가입 </h2>
      <ul>
        <li>
          <label>
            아이디 :
            <input
              type="text"
              placeholder="아이디"
              value={userid}
            />
          </label>
        </li>
        <li>
          <label>
            비밀번호 :
            <input
              type="password"
              placeholder="비밀번호"
              value={userpw}
            />
          </label>
        </li>
      </ul>
    </div>
  );
}
```

```
<li>
  <label>
    이름 :
    <input
      type="text"
      placeholder="이름"
      value={username}
    />
  </label>
</li>
</ul>
<div className="btn">
  <button type="submit">
    회원가입
  </button>
</div>
</div>
);
}
```



```
.joinWrap {  
  max-width: 400px;  
  margin: 2rem auto;  
  padding: 2rem;  
  background-color: #fff;  
  border-radius: 8px;  
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.1);  
}
```

```
.joinWrap h2 {  
  text-align: center;  
  margin-bottom: 1.5rem;  
  color: #333;  
}
```

```
.joinWrap ul {  
  list-style: none;  
  padding: 0;  
  margin: 0;  
}
```

```
.joinWrap li {  
  margin-bottom: 1rem;  
}
```

```
.joinWrap label {  
  display: flex;  
  align-items: center;  
  gap: 0.5rem;  
  color: #555;  
  font-size: 0.9rem;  
}
```

```
.joinWrap input {  
  flex: 1;  
  padding: 0.7rem;  
  border: 1px solid #ddd;  
  border-radius: 4px;  
  font-size: 1rem;  
  box-sizing: border-box;  
}
```

```
.joinWrap input:focus {  
  outline: none;  
  border-color: #0066cc;  
}
```

```
.joinWrap .btn {  
  margin-top: 1.5rem;  
}
```

```
.joinWrap button {  
  width: 100%;  
  padding: 0.8rem;  
  background-color: #0066cc;  
  color: #fff;  
  border: none;  
  border-radius: 4px;  
  font-size: 1rem;  
  cursor: pointer;  
  transition: background-color 0.2s;  
}
```

```
.joinWrap button:hover {  
  background-color: #0052a3;  
}
```



React 쇼핑몰

Home

Join

회원가입

아이디 : 아이디

비밀번호 : 비밀번호

이름 : 이름

회원가입



➤ PHP 내장 명령어

코드	작명(✓)	내장(○)	설명
include		○	파일 포함 명령어
\$data	✓		변수명
\$userid	✓		변수명
json_decode()		○	JSON→배열 변환 함수
\$conn	✓		변수명 (DB 연결 객체)
prepare()		○	SQL 준비 메서드
bind_param()		○	데이터 바인딩 메서드
execute()		○	SQL 실행 메서드
get_result()		○	결과 가져오기 메서드
num_rows		○	결과 행 개수 속성
"status"	✓		JSON 키 이름
users (테이블)	✓		DB 테이블명



➤ PHP 기본 문법

- PHP의 연관배열 작성하는 방법

언어	문법	의미	예시
JavaScript(React)	: (콜론)	키와 값 연결	{name: "Kim"}
PHP	=> (이중화살표)	키와 값 연결	["name" => "Kim"]
공통	= (등호 1개)	변수에 값 대입	\$x = 10

JavaScript 객체

```
key : value
```



```
{name: "Kim"}
```

PHP 연관 배열

```
key => value
```



```
["name" => "Kim"]
```

- PHP의 객체의 속성(property)에 접근 하는 방법

언어	문법	예시
JavaScript(React)	.	conn.connect_error
PHP	-> (화살표)	\$conn->connect_error



➤ Join-project -> backend -> config -> database.php 에 아래 내용을 코딩한다.

- MySQL DB를 PHP에 연결한다.

```
<?php
//CORS 설정(React에서 접근 가능하도록 지정)
header("Access-Control-Allow-Origin: http://localhost:5173");
header("Access-Control-Allow-Methods: POST, GET, OPTIONS");
header("Access-Control-Allow-Headers: Content-Type");
header("Content-Type: application/json; charset=UTF-8");

// OPTIONS 요청 처리 (CORS preflight)
if ($_SERVER['REQUEST_METHOD'] === 'OPTIONS') {
    http_response_code(200);
    exit();
}

//데이터베이스 연결정보
define('db_host','localhost');
define('db_user','root'); //학원은 green
define('db_pass',''); //XAMPP는 기본값 비밀번호 없음
define('db_name','phpjoin');
```

```
//데이터베이스 연결함수
function getDBConnection(){
    $conn = new mysqli(db_host,db_user,db_pass,db_name);

    // 연결 오류 체크
    if ($conn->connect_error) {
        die(json_encode([
            "status" => "fail",
            "message" => "DB 연결 실패: " . $conn->connect_error
        ]));
    }

    // UTF-8 설정
    $conn->set_charset("utf8mb4");
    return $conn;
}

// 연결 객체 생성 (include 시 바로 사용 가능하도록)
$conn = getDBConnection();
?>
```



➤ Join-project -> backend -> api -> join.php 에 아래 내용을 코딩한다.

- 연결한 DB에 SELECT, INSERT를 한다.

```
<?php
// DB 연결이 들어있는 config의 database.php를 포함
include '../config/database.php';

// -----
// 클라이언트에서 보낸 JSON 데이터 읽기
// -----
// React나 다른 프론트엔드에서 fetch/axios로 보낸 POST 데이터를
// php://input을 통해 읽고, json_decode로 배열로 변환
$data = json_decode(file_get_contents("php://input"), true);

// -----
// 데이터 유효성 체크
// -----
// userid, userpw, username이 없으면 오류 반환 후 종료
if (!$data || !isset($data['userid'], $data['userpw'], $data['username'])) {
    echo json_encode([
        "status" => "fail",
        "message" => "필수 데이터가 없습니다." // 클라이언트에게 메시지 전달
    ]);
    exit;
}
```



➤ Join-project -> backend -> api -> join.php 에 아래 내용을 코딩한다.

```
// 입력 데이터 변수에 저장
$userid = $data['userid'];
$username = $data['username'];
$userpw = $data['userpw'];

// 빈 값 체크
if (empty($userid) || empty($username) || empty($userpw)) {
    echo json_encode([
        "status" => "fail",
        "message" => "모든 필드를 입력해주세요."
    ]);
    exit;
}

// 중복 아이디 확인
$stmt = $conn->prepare("SELECT * FROM users WHERE userid=?");
$stmt->bind_param("s", $userid);
$stmt->execute();
$result = $stmt->get_result();

if ($result->num_rows > 0) {
    echo json_encode(["status"=>"fail", "message"=>"이미 존재하는 아이디입니다."]);
    $stmt->close();
    $conn->close();
    exit;
}

// Prepared Statement를 사용한 안전한 INSERT
// SQL 인젝션 공격 방지
$sql = "INSERT INTO users (userid, username,userpw) VALUES (?, ?, ?)";
$stmt = $conn->prepare($sql);
$stmt->bind_param("sss", $userid, $username, $userpw);
// "sss" : 세 개의 문자열(string) 데이터를 바인딩
```



➤ Join-project -> backend -> api -> join.php 에 아래 내용을 코딩한다.

```
// 실행 후 결과 확인
$response = [];

if ($stmt->execute()) {
    // INSERT 성공
    $response["status"] = "success";
    $response["message"] = "회원가입이 완료되었습니다.";
} else {
    // INSERT 실패 (예: 이미 존재하는 userid)
    $response["status"] = "fail";
    $response["message"] = "회원가입 실패: " . $stmt->error;
}

// 클라이언트에게 JSON으로 결과 전송
echo json_encode($response);

// 사용한 Statement와 DB 연결 종료
$stmt->close();
$conn->close();
?>
```



➤ 회원 가입 후 DB 출력 화면

phpMyAdmin

최근 즐겨찾기

- 새로운
- information_schema
- mysql
- performance_schema
- phpjoin
 - New
 - users
- phpmyadmin
- test

서버: 127.0.0.1 » 데이터베이스: phpjoin » 테이블: users

보기 구조 SQL 검색 삽입 내보내기 가져오기 권한

0 - 1행 표시 중 (합계 2, 질의 실행시간 0.0001 초.)

`SELECT * FROM `users`;`

☐ 프로파일링 [인라인 편집] [수정] [SQL 해석] [PHP 코드 보기] [다시 보기]

☐ 모두 보기 | 행 갯수: 25 | 행 필터링: 현재 테이블 검색 | Sort by key:

Extra options

	userid	username	userpw
☐ 수정 복사 삭제	aaa	개나리	2222
☐ 수정 복사 삭제	kkk	홍길동	1111

로그인 JSX





```
import './LoginForm.css';

export default function LoginForm() {

  return (
    <div className="loginWrap">
      <h2>로그인</h2>
      <form >
        <ul>
          <li>
            <label>
              아이디 :
            <input
              type="text"
              placeholder="아이디"
            />
            </label>
          </li>
          <li>
            <label>
              비밀번호 :
            <input
              type="password"
              placeholder="비밀번호"
            />
            </label>
          </li>
        </ul>
        <div className="btn">
          <button type="submit">로그인</button>
        </div>
      </form>
    </div>
  );
}
```

```
.loginWrap {
  max-width: 400px;
  margin: 50px auto;
  padding: 30px;
  border: 1px solid #ddd;
  border-radius: 8px;
  background-color: #fff;
  box-shadow: 0 2px 10px rgba(0, 0, 0, 0.1);
}

.loginWrap h2 {
  text-align: center;
  margin-bottom: 30px;
  color: #333;
}

.loginWrap ul {
  list-style: none;
  padding: 0;
  margin: 0;
}

.loginWrap li {
  margin-bottom: 20px;
}
```



```
.loginWrap label {
  display: flex;
  gap: 10px;
  font-weight: bold;
  color: #555;
  margin-bottom: 5px;
}

.loginWrap input {
  flex:1;
  width: 100%;
  padding: 10px;
  border: 1px solid #ddd;
  border-radius: 4px;
  font-size: 14px;
  box-sizing: border-box;
}

.loginWrap input:focus {
  outline: none;
  border-color: #4CAF50;
}

.loginWrap .btn {
  text-align: center;
  margin-top: 20px;
}
```

```
.loginWrap button {
  width: 100%;
  padding: 12px;
  background-color: #4CAF50;
  color: white;
  border: none;
  border-radius: 4px;
  font-size: 16px;
  cursor: pointer;
  transition: background-color 0.3s;
}

.loginWrap button:hover {
  background-color: #45a049;
}
```



로그인

아이디 :

비밀번호 :

로그인



➤ Join-project -> backend -> api -> login.php 에 아래 내용을 코딩한다.

- 연결한 DB에서 userid, userpw를 검색해 로그인한다.

```
<?php
include '../config/database.php';

$data = json_decode(file_get_contents('php://input'),true);

$userid = $data['userid'];
$userpw = $data['userpw'];

// 빈값 체크
if(empty($userid) || empty($userpw)){
    echo json_encode([
        'status' => 'fail', 'message' => '아이디,비밀번호 모두 입력'
    ]);
    exit;
}

$sql = 'SELECT userid, userpw FROM users WHERE userid=? AND userpw=?';
$stmt= $conn->prepare($sql);
$stmt->bind_param('ss',$userid,$userpw);
$stmt->execute();
$result = $stmt->get_result();
```

```
$responses = [];
if($result->num_rows > 0){
    $responses['status'] = 'sucess';
    $responses['message'] = '로그인 성공';
    // 로그인 성공 시 userid를 아래처럼 포함 시켜야
    // LoginForm.jsx에서 userid를 화면에 출력할 수 있다.
    $responses['userid'] = $userid;
}else{
    $responses['status']='fail';
    $responses['message']='아이디 또는 비밀번호가 일치하지 않습니다.';
}
echo json_encode($responses);
$stmt->close();
$conn->close();

?>
```



➤ 로그인 한 결과 화면

localhost:5173/login

localhost:5173 내용:
kkk님 환영합니다.

확인

로그인

아이디 : kkk

비밀번호 :

로그인

react-dom_client.js?v=7071a6bf:20103
Download the React DevTools for a better development experience: <https://react.dev/link/react-devtools>

로그인 LoginForm.jsx:18
▶ {status: 'sucess', message: '로그인 성공', userid: 'kkk'}



로그인 한 결과 화면

React 쇼핑몰

Home Login Join

여기는 홈화면 입니다.

요소 콘솔 소스 네트워크 >> 2

top 필터 기본 수준

문제 2건: 2 3개 숨김

☐ 네트워크 숨기기 ☐ XMLHttpRequest 기록

☐ 로그 보존 ☒ 적극적 평가

☐ 선택된 컨텍스트만 ☒ 기록에서 자동 완성

☒ 콘솔에서 유사한 메시지 그룹화 ☒ 코드 평가를 사용자 작업으로 취급

☒ 콘솔에 CORS 오류 표시

[react-dom_client.js?v=7071a6bf:20103](#)

Download the React DevTools for a better development experience:
<https://react.dev/link/react-devtools>

로그인 [LoginForm.jsx:18](#)

```
{status: 'sucess', message: '로그인 성공', userid: 'kkk'} i
  message: "로그인 성공"
  status: "sucess"
  userid: "kkk"
  ▶ [[Prototype]]: Object
```

세션(session) 이란





➤ 세션(session) vs 기존 방법 비교

- 보안성: 중요한 정보가 서버에만 있음
- 조작 불가: 클라이언트에서 세션 정보를 직접 수정할 수 없음
- 서버 제어: 서버에서 언제든지 세션을 무효화 가능
- 실무 표준: 대부분의 웹사이트가 사용하는 방식

방법	저장 위치	보안성	설명
LocalStorage	클라이언트(브라우저)	낮음	JS로 쉽게 접근 가능, XSS 공격에 취약
Cookie	클라이언트(브라우저)	중간	HttpOnly 설정 시 JS 접근 불가
Context API	메모리(임시)	-	새로고침 시 사라짐 (보통 LocalStorage와 함께 사용)
Session	서버	높음	서버에서 관리, 클라이언트는 세션ID만 가짐

특징	LocalStorage	Cookie	Session (서버)	Context API
저장 위치	브라우저	브라우저	서버	React 메모리
지속성	영구	설정 가능	로그아웃/만료시	새로고침시 삭제
용량	5-10MB	4KB	제한 없음	메모리 제한
서버 접근	X	✓	✓	X
보안	낮음	중간	높음	높음
사용자 접근	✓가능	✓가능	X 불가능	✓가능



➤ 로그인 상태 유지 : Context API + LocalStorage

- 실무에서는 보안 때문에 로그인 상태를 Session으로 개발한다.
- 단, 지금은 프론트엔트 개발부분이기때문에 PHP + MySQL 을 연동하여 가입한 회원의 정보를 Context API + LocalStorage로 로그인 상태유지를 개발하기로 한다.

➤ Join-project -> src -> context -> AuthContext.jsx 을 생성 하고 아래 코드를 작성한다.

```
import { createContext, useContext, useState, useEffect } from 'react';

export const AuthContext = createContext();

export default function AuthProvider({ children }) {
  const [userid, setUserId] = useState(null);

  // 컴포넌트 마운트 localStorage에서 사용자 정보 불러오기
  useEffect(() => {
    const savedUserId = localStorage.getItem('userid');
    if (savedUserId) {
      setUserId(JSON.parse(savedUserId));
      //setUserId(savedUserId);
    }
  }, []);
```

```
// 로그인
// 저장 (문자열만 가능)
// localStorage.setItem('키이름', '저장할값');
// localStorage.setItem('userid', res.data.userid);
const loginsave = (userData) => {
  setUserId(userData);
  localStorage.setItem('userid', JSON.stringify(userData));
};

// 로그아웃
const logout = () => {
  setUserId(null);
  localStorage.removeItem('userid');
};

return (
  <AuthContext.Provider value={{ userid, loginsave, logout }}>
    {children}
  </AuthContext.Provider>
);
}
```



➤ Join-project -> src -> join -> loginFrom.jsx 에 아래 내용을 기존코드에 추가하여 코딩한다.

```
import './LoginForm.css';
import { useState } from 'react';
import axios from 'axios';
import { useNavigate } from 'react-router-dom';
import { useContext } from 'react';
import { AuthContext } from '../context/AuthContext';

export default function LoginForm() {
  const [userid, setUserid] = useState("");
  const [userpw, setUserpw] = useState("");
  const navigate = useNavigate();
  const { loginsave } = useContext(AuthContext); //
  Context API 이용 데이터 공유
```

```
const login = async (e) => {
  e.preventDefault();
  try {
    const res = await axios.post(
      'http://localhost/join-project/backend/api/login.php',
      { userid, userpw }
    );
    console.log('로그인', res.data);
    if (res.data.status === 'success') {
      // ★ Context에 저장해야 Header가 갱신됨!
      loginsave(res.data.userid);
      alert(`${res.data.userid}님 환영합니다.`);
      navigate('/');
      setUserid("");
      setUserpw("");
    } else {
      alert(res.data.message || '로그인 실패');
    }
  } catch (error) {
    console.log('에러', error);
    alert('서버 연결 오류');
  }
};
```



➤ Join-project -> src -> join -> Header.jsx 에 아래 내용을 기존코드에 추가하여 코딩한다.

```
import { useEffect, useState } from 'react';
import { Link } from 'react-router-dom';
import { useNavigate } from 'react-router-dom';
import './Header.css';
import { useContext } from 'react';
import { AuthContext } from '../context/AuthContext';

export default function Header() {
  const navigate = useNavigate();
  const { userid, logout } = useContext(AuthContext);

  // 로그아웃 핸들러
  const logoutHandler = () => {
    logout();
    alert('로그아웃 되었습니다.');
```

로그아웃 버튼을 클릭하면 alert를 띄우고, navigate('/')를 호출하여 홈으로 이동한다.

```
    navigate('/');
  };
}
```

```
return (
  <header className="header">
    <h2>
      <Link to="/" className="logo">
        { ' ' }
        React 쇼핑몰
      </Link>
    </h2>
    <nav className="nav">
      <Link to="/">Home</Link>
      {userid ? (
        // 로그인 상태
        <>
          <span>{userid}님</span>
          <button type="button" onClick={logoutHandler}>
            Logout
          </button>
        </>
      ) : (
        // 로그아웃 상태
        <>
          <Link to="/login">Login</Link>
          <Link to="/join">Join</Link>
        </>
      )}
    </nav>
  </header>
);
}
```



➤ 로그인 후 Header인 HOME으로 이동 => userid 님 로그아웃 출력 화면

